Stability by Design: Optimizing Prompt Properties in Small Language Models via Sensitivity Metrics*

Yaniv Mualem (ID: 209127612)
yanivmualem@mail.tau.ac.il

Sharon Levy (ID: 315140152)
sharon14@mail.tau.ac.il

Eden Daya (ID: 315285759)
edendaya@mail.tau.ac.il

Avner Fivelovich (ID: 314948811)
Avnerf@mail.tau.ac.il

Moodle Group Number: 14

Abstract

Prompting exerts a significant influence on small language models (SLMs), though this impact is not always discernible through task accuracy alone. This study investigates prompt robustness by analyzing input sensitivity under semantic-preserving perturbations. Specifically, it examines whether three prompt properties - metacognition, structure, and politeness - systematically affect model behavior. Using an inference-only framework, evaluations were conducted on the CoLA, QASC, and CSQA datasets using locally runnable models. These models span both encoder-decoder and decoder-only architectures, including base and instruction-tuned variants. The methodology distinguishes baseline task competence from sensitivity to ensure that consistent incorrectness is not misinterpreted as functional stability. Final results indicate a strong interaction between prompt style, model capability, and task type. Structured prompting provides the clearest benefits on CoLA but can sharply reduce accuracy on multiple-choice tasks. Politeness serves as the most consistently safe intervention, while metacognitive prompting demonstrates the highest levels of instability. These findings suggest that prompt robustness in SLMs must be evaluated through a joint framework of task competence, parseability, and output behavior, rather than through sensitivity metrics in isolation.

1 Introduction

Small language models (SLMs) are increasingly attractive for practical NLP deployment because they are cheaper to run, easier to host locally, and more suitable for privacy-sensitive or resource-constrained settings than frontier-scale models. At the same time, these practical advantages come with a limitation: model behavior can depend strongly on how a task is phrased. In practice, small changes to a prompt, such as synonym substitutions, paraphrases, added conversational phrasing, or altered output formats, may change the model's output and, as a result, its measured task performance.

This matters both scientifically and practically. From a scientific perspective, prompt instability makes evaluation harder to interpret. If two prompts express the same task intent but lead to different outputs, then part of the observed performance may reflect wording choices rather than the model's actual ability. From a practical perspective, instability reduces reliability. In real applications, prompts are often reformulated by users, developers, or other components in a pipeline. Ideally, these reformulations should not cause large and unpredictable changes in model behavior. This concern is especially relevant for SLMs, where limited capacity and weaker instruction-following may amplify the effect of superficial prompt variation.

Sensitivity, however, is not meaningful on its own. A model that gives the same incorrect answer across all prompt variants may appear stable even though it is not robust in any useful sense. For this reason, prompt robustness should be studied together with baseline task competence. In this paper, we study prompt robustness in SLMs through the lens of **input sensitivity**, while also checking whether a model performs the underlying task well enough for sensitivity scores to be interpretable.

Following Lu et al. (2024), we define prompt sensitivity as the extent to which a model's output changes when the original prompt is replaced with lightly perturbed variants that preserve its meaning. We adapt this framework to small, locally runnable models and use a controlled perturbation pipeline that generates semantic-preserving variants through synonym replacement and paraphrasing. Our goal is not only to measure instability,

*Code available at https://github.com/yanivmu/NLP_Stability_by_Design

but also to ask whether particular prompt properties systematically affect it.

We focus on three prompt properties inspired by prior work on prompt design (Long et al., 2025). The first is **metacognition**, implemented through self-check or self-verification cues. The second is **structure**, implemented through explicit output constraints, in our case a fixed JSON response format. The third is **politeness**, implemented through courteous conversational phrasing. These conditions are compared against a standard zero-shot control prompt.

Research question: Do certain prompt properties, metacognition, structure, and politeness, make language models more stable when inputs undergo semantic-preserving perturbations?

To answer this question, we use an inference-only evaluation framework. For each prompt style, we run the same model on the original prompt and on several semantic-preserving perturbations, and we measure sensitivity using the variation ratio,

$$s = 1 - \frac{f_m}{N+1},$$

where f_m is the frequency of the modal output across the original prompt and its N perturbed variants. In the main setting of our implementation, this metric is computed over raw decoded outputs after minimal normalization, while task accuracy is computed over parsed task labels. Lower values indicate greater consistency across perturbations. We evaluate this framework on three benchmarks with different task structures: CoLA, a binary grammatical acceptability task; QASC, an 8-way multiple-choice science question answering task; and CSQA, a 5-way multiple-choice commonsense reasoning task. We test a suite of locally runnable models that includes both encoder-decoder and decoder-only architectures, as well as base and instruction-tuned variants.

Our contributions are as follows:

- We formulate prompt robustness in SLMs as a measurable sensitivity problem, while emphasizing that sensitivity is only interpretable when the model shows non-trivial baseline task competence.
- We adapt a sensitivity-based evaluation framework to small, locally runnable models using a controlled perturbation pipeline based on semantic-preserving synonym replacement and paraphrasing.
- We analyze how metacognition, structure, and politeness affect both sensitivity and task performance across three benchmark settings with different output spaces and difficulty profiles.

2 Related Work

Our work connects two lines of research: work on **prompt design and prompt attributes**, and work on **prompt sensitivity and input stability**.

The first line of work asks what makes a prompt effective. Recent research has moved beyond treating prompts as isolated heuristics and instead describes them in terms of reusable and interpretable attributes. In particular, Long et al. (2025) propose a taxonomy of prompt attributes that organizes prompt design choices into dimensions such as metacognitive guidance, structural constraints, and stylistic framing. This perspective is central to our study because it makes prompt design easier to analyze systematically rather than only through trial and error. We build directly on this view by selecting three attributes, metacognition, structure, and politeness, and operationalizing them in a controlled evaluation setup.

The second line of work studies prompt quality through robustness to input variation. Lu et al. (2024) analyze prompts in terms of **input stability**, showing that prompts differ substantially in their sensitivity to minor perturbations and that higher sensitivity often accompanies worse downstream performance. This work motivates our study in two ways. First, it suggests that prompt evaluation should consider not only accuracy, but also whether outputs remain stable under semantically equivalent reformulations. Second, it provides a concrete sensitivity metric, variation ratio, that can serve as a diagnostic signal during prompt evaluation.

Our work builds on both lines of research, but it also differs from them in several important ways. Compared with prior work on prompt attributes, we are not only interested in whether a prompt improves average performance, but also in whether it changes robustness under controlled perturbations. Compared with prior work on sensitivity, we focus specifically on **small, locally runnable models**, including both base and instruction-tuned variants, where prompt sensitivity may interact strongly with limited instruction-following ability and with the model's underlying task competence.

In this setting, low sensitivity is not always desirable if it simply reflects consistently wrong behavior. For this reason, our framework separates baseline task competence from sensitivity analysis by checking out-of-the-box accuracy before interpreting robustness scores.

The gap we address lies in the connection between **how prompts differ in form** and **how prompts differ in robustness**. Work on prompt attributes explains how prompts can vary in structure, intention, and style. Work on sensitivity shows that prompts can vary in stability under perturbation. What remains less explored is the relation between these two questions in small-model settings: which prompt properties make prompts more or less stable, when greater stability aligns with better task performance, and when stability can become a misleading signal. We address this gap by combining an attribute-based view of prompting with a sensitivity-based evaluation framework.

3 Method

3.1 Task Definition

We study prompt robustness as a conditional prediction problem under semantic-preserving reformulation. Let x denote an input instance, let y denote its gold label, and let p denote a prompt style. A model receives a prompted version of the input, produces a prediction $\hat{y}$, and is then evaluated both on the original input and on lightly perturbed variants of that same input.

Our setting includes three task types. In QASC, the input is a multiple-choice science question with eight answer options, and the target output is a single answer letter from A to H. In CoLA, the input is a sentence and the target output is a binary grammaticality judgment, represented as Yes or No. In CSQA, the input is a multiple-choice commonsense question with five answer options, and the target output is a single answer letter from A to E. In all cases, the task content remains fixed across prompt styles; only the framing of the instruction changes. Detailed statistics and baseline performance for each dataset are summarized in Table 1.

For each instance, we compare four prompt styles. The **control** condition is a standard zero-shot instruction. The **metacognition** condition adds self-check and verification cues. The **structure** condition constrains the model to respond in a fixed JSON format that separates a brief explanation from a final answer field. The **politeness** condition adds courteous conversational phrasing. The goal is not to optimize prompts for a single benchmark, but to test whether these prompt properties change the stability of model predictions under controlled reformulations of the same underlying input.

Dataset	Task Type	Domain	Split	N_{val}	Majority	Random
CoLA	Binary Class.	Grammar	Val	1043	69.1%	50.0%
QASC	8-way MC	Science	Val	926	14.6%	12.5%
CSQA	5-way MC	Commonsense	Val	1221	20.9%	20.0%

Table 1: Statistics for the evaluation datasets. N_{val} denotes the number of examples in the validation split. Majority denotes the accuracy of the majority-class baseline.

3.2 Approach

Our framework is inference-only. We do not fine-tune any model parameters. Instead, we evaluate how the same pretrained model behaves when the same task is presented through different prompt styles and through semantic-preserving input perturbations. The overall evaluation flow is summarized in Algorithm 1.

Algorithm 1: Sensitivity Evaluation Pipeline

Input: Dataset D, Model M, Style S

1. OOTB Screening:
Evaluate M on D using Control prompt.
If Accuracy $<$ Threshold: **Exit**.

2. Perturbation:
For each item $x \in D$:
Generate N semantic variants $\{x'_1, ..., x'_N\}$.

3. Inference:
Collect responses $R = \{M(S, x), M(S, x'_1), ..., M(S, x'_N)\}$.

4. Metric Calculation:
Calculate Sensitivity $s = 1 - \frac{modal\_count(R)}{N+1}$.
Parse R to calculate Accuracy a.

Figure 1: The sensitivity-based evaluation process.

For each model-dataset pair, we first run an out-of-the-box (OOTB) accuracy check using the control prompt. This step verifies that the model performs the task at a level that makes sensitivity analysis meaningful. Only after this screening step do we run the full sensitivity experiment.

In the main experiment, we sample input instances and evaluate each prompt style separately. For every sampled instance, we construct one original prompt and N perturbed variants. The model is run on all $N + 1$ versions. We then record both the raw decoded outputs and the parsed task-level

Style	Example Prompt (CoLA)
Control	Is this sentence grammatically correct? Answer Yes or No. Sentence: "The problem perceives easily." Answer:
Metacognition	Is this sentence grammatically correct? Answer Yes or No. Before answering, carefully check the grammar rules. Verify your answer is correct. Sentence: "The problem perceives easily." Think about it carefully, then answer:
Structure	Analyze whether the following sentence is grammatically correct. Sentence: "The problem perceives easily." You MUST respond with valid JSON in this exact format: {"reasoning": "your brief explanation here", "final_answer": "Yes or No"}
Politeness	Hello! I would really appreciate your help with this. Could you please tell me if this sentence is grammatically correct? Please answer with Yes or No. Sentence: "The problem perceives easily." Thank you! Your answer:

Table 2: Examples of the four prompt styles applied to a single instance from the CoLA dataset.

answers. For QASC and CSQA, parsing extracts a single answer letter. For CoLA, parsing extracts a Yes or No judgment. In the structure condition, the parser reads the `final_answer` field from the JSON output; in the other conditions, it extracts the relevant label from plain-text output.

We use two perturbation strategies. The default strategy is synonym-based perturbation. The input text is tokenized and part-of-speech tagged, and candidate replacement words are restricted to content words such as nouns, verbs, adjectives, and adverbs. Very short words, stopwords, question words, and answer labels are excluded. Candidate synonyms are taken from WordNet, with preference for the most common senses in order to reduce semantic drift. When synonym replacement cannot produce enough distinct variants, the pipeline uses lightweight fallback transformations such as minor article changes, filler prefixes, or punctuation variation. For example, a CoLA sentence such as *"The problem perceives easily."* might be perturbed to *"The trouble perceives easily."* (synonym: problem → trouble), while a QASC question such as *"What do plants need to grow?"* might become *"What do plants require to grow?"* (synonym: need → require).

The second strategy is paraphrasing. Here, a separate small model is used to rewrite the input while preserving its meaning. The paraphrase generator cycles through multiple rewrite instructions and uses stochastic decoding to produce diverse variants. Duplicate paraphrases are removed, and any shortfall is again completed by the synonym-based fallback. In both perturbation modes, the final goal is the same: produce multiple variants that change surface form while preserving task intent as much as possible.

For each instance and prompt style, we compute sensitivity using the variation ratio over the model's raw decoded outputs after minimal normalization. Adapting the variation ratio metric from Lu et al. (2024), we evaluate sensitivity on raw decoded text rather than parsed task labels. This deliberate choice captures not only label flips but also instability in formatting, hedging, and output style, which are critical components of prompt robustness in small models. In parallel, we compute task accuracy over parsed task labels. This separation lets us distinguish instability in generation from correctness on the task, and avoids treating repeated wrong answers as meaningful robustness.

4 Experimental Setup

4.1 Data

The framework is evaluated on three benchmarks with different task structures.

The first is QASC, an 8-way multiple-choice science question answering benchmark. Each example contains a question and eight candidate answers. To isolate reading comprehension bounds and avoid ceiling effects, QASC is evaluated exclusively on questions and answer choices, without supporting facts. This setup ensures that model behavior reflects prompt-driven reasoning rather than trivial fact-retrieval, making the benchmark more suitable for studying whether prompt style changes both robustness and performance under a non-trivial level of difficulty.

The second benchmark is CoLA, a binary grammatical acceptability task. Each example consists

of a single sentence, and the model must decide whether it is grammatically acceptable. CoLA complements the multiple-choice datasets in two useful ways. First, it has a much simpler output space. Second, it evaluates a different kind of linguistic competence, grammatical judgment rather than answer selection among explicit alternatives.

The third benchmark is CommonsenseQA (CSQA), a 5-way multiple-choice commonsense reasoning task. Each example contains a question and five candidate answers. CSQA is included to extend the multiple-choice setting beyond QASC and test whether prompt effects generalize to a different reasoning domain. Together, QASC and CSQA enable a comparison between two multiple-choice tasks with different knowledge demands, while CoLA provides a contrasting binary setting.

All experiments use the validation split of each benchmark. The final reported results are aggregated over repeated runs across seeds and perturbation settings under the finalized evaluation setup.

4.2 Models

A set of locally runnable models is evaluated, chosen to vary along two dimensions: architecture and instruction tuning. The model suite includes both encoder-decoder and decoder-only systems, as well as both base and instruction-tuned variants.

The encoder-decoder models are Flan-T5-Base and Flan-T5-Large. These models are useful reference points because they are explicitly instruction-tuned and relatively strong on zero-shot text-to-text tasks. The decoder-only base models are Pythia-410M and Llama-3.2-1B. These models enable an examination of whether prompt sensitivity behaves differently when instruction-following ability is limited. The instruction-tuned decoder-only models are Llama-3.2-1B-Instruct and Phi-3-Mini-4K-Instruct. These provide a comparison point within the decoder-only family between base and instruction-tuned behavior.

All models are loaded directly from HuggingFace and evaluated without any parameter updates. For causal models, padding is set on the left and the end-of-sequence token is used as a padding token when necessary. For instruction-tuned chat models, the framework automatically applies the tokenizer's chat template before generation. This keeps model-specific formatting separate from the core evaluation loop and ensures that each model is used in a way that is consistent with its intended interface.

4.3 Implementation Details

As our study focuses on prompt sensitivity rather than adaptation or fine-tuning, there is no training stage. All experiments are performed with frozen pretrained models and deterministic inference.

Generation is run with greedy decoding so that differences in output are driven by prompt and input changes rather than by sampling noise in the evaluated model itself. For each dataset, we sample 500 instances, and for each instance we generate $N = 20$ semantic-preserving perturbations. The maximum number of generated tokens is task- and style-dependent. Plain answer formats use shorter generation limits, while structured prompts use longer budgets because the model must produce a valid JSON object. Metacognitive prompts on CoLA are also allowed longer outputs so that self-check instructions are not systematically truncated before the model reaches a final answer.

The implementation is designed for reproducibility. Random seeds are fixed across Python, NumPy, PyTorch, and the Transformers library. The framework uses three experiment seeds (105, 2266, and 86379), and each run is fully determined by its seed and configuration. The perturbation pipeline also uses seed control at the level of individual examples so that the same configuration reproduces the same perturbation set across runs. Models are executed on the best available device, including CUDA, Apple MPS, or CPU, and half precision is used for selected decoder-only models when supported to improve the inference speed.

5 Evaluation

Each model-prompt combination is evaluated along two axes: task performance and sensitivity.

The first metric is **OOTB accuracy**. Before running sensitivity experiments, the model's accuracy on unperturbed inputs is measured using the control prompt. This serves as a screening step. If a model performs at or near random guessing, then sensitivity is not informative, because the model may simply be repeating the same wrong answer. In the final implementation, OOTB accuracy is computed over all sampled items rather than only over successfully parsed responses, so unparseable responses count as incorrect. The screening thresholds are dataset-specific. For QASC, the random baseline is 12.5% and the valid-signal threshold is 40%. For CoLA, the random baseline is 50% and the valid-signal threshold

is 60%. For CSQA, the random baseline is 20% and the valid-signal threshold is 40%.

The second metric is **variation ratio**, which measures the dispersion of outputs across the original input and its perturbations:

$$s = 1 - \frac{f_m}{N+1},$$

where f_m is the frequency of the modal output across the original prompt and its N perturbed variants. Lower values indicate greater consistency, while higher values indicate greater sensitivity to reformulation. In the main setting of the experiments, this score is computed over raw decoded outputs after minimal normalization rather than over parsed task labels. This design choice treats changes in generated form as part of prompt instability, which is especially relevant for structured prompts and for instruction-following settings where formatting itself is part of the response behavior. At the same time, task accuracy is kept separate and is always computed over parsed task labels.

In addition to average variation ratio, **style-specific accuracy** is reported on the original, unperturbed inputs used in the main experiment. This lets prompt styles be compared not only in terms of stability, but also in terms of whether increased consistency coincides with better or worse task performance. Together, these metrics allow the distinction of several cases: prompts that are both accurate and stable, prompts that are accurate but unstable, prompts that are stable only at the level of output format, and prompts that appear stable only because the model gives the same incorrect answer repeatedly.

6 Results

Results obtained with the finalized evaluation pipeline described above are reported here. The values below summarize the final aggregated result pool.

Table 3 shows that model capability varies sharply by task. Flan-T5-Large and Phi-3-Mini are the only models that perform solidly across all three datasets. Flan-T5-Base performs well on the multiple-choice tasks but fails on CoLA. The smaller base decoder-only models (Llama-3.2-1B and Pythia-410M) perform competitively on CoLA but collapse on QASC and CSQA. This likely reflects the fact that these models are not instruction-tuned: they can handle CoLA's simple binary format but struggle to select and output a single answer letter from multiple choices, often producing text continuations instead of valid answers. This confirms that low sensitivity on weak model-task pairs would be difficult to interpret as genuine robustness.

Model	CoLA	QASC	CSQA
Phi-3-Mini	77.0	74.0	69.0
Flan-T5-Large	62.0	62.0	80.0
Flan-T5-Base	30.7	42.0	74.0
Llama-3.2-1B	70.0	0.0	1.0
Llama-3.2-1B-Instruct	70.0	11.0	19.0
Pythia-410M	65.0	11.0	17.0

Table 3: OOTB accuracy (%) across all model-dataset pairs under the final evaluation setup.

Although both models succeeded across all datasets, the high Variation Ratio (VR) of Phi-3-Mini makes its results less instructive. Therefore, we focus on Flan-T5-Large as the main cross-task comparison in Table 4.

Table 4 demonstrates that the effectiveness of a prompt style is highly dependent on the underlying task structure. The most striking pattern is the split between CoLA and the multiple-choice tasks: every non-control style improves accuracy on CoLA, but the same styles hurt performance on CSQA and QASC. This suggests that CoLA's simple binary output space is more forgiving of prompt modifications, while multiple-choice tasks are brittle when the prompt adds complexity.

The multiple-choice tasks (CSQA and QASC) show that complex prompts can be actively destructive. While Politeness remains a safe and effective approach as the only style that never degrades accuracy on any task, both Metacognition and Structure cause massive performance degradations, likely because the added reasoning instructions cause the model to generate longer, more variable outputs that drift from the requested answer format.

The most crucial takeaway emerges from the QASC task: applying the Structure prompt severely crashes the accuracy (dropping to 22.1%), yet it appears highly 'stable' due to a reduced variation ratio. This exemplifies the 'misleading stability' discussed in the introduction.

Figure 2 provides a high-level overview of the trade-offs associated with each prompt attribute. While specific settings like CoLA show perfor-

	CoLA		CSQA		QASC	
Style	Acc. (%)	VR	Acc. (%)	VR	Acc. (%)	VR
Control	63.7	0.1621	83.2	0.2242	62.2	0.2345
Metacognition	67.7	0.1693	59.9	0.7021	36.9	0.6529
Structure	70.3	0.1344	50.4	0.2237	22.1	0.1428
Politeness	69.9	0.1778	83.4	0.2170	62.2	0.2377

Table 4: Prompt-style comparison for Flan-T5-Large across the three main benchmarks. Acc. is task accuracy; VR is average variation ratio. Values are aggregated over the final result pool.

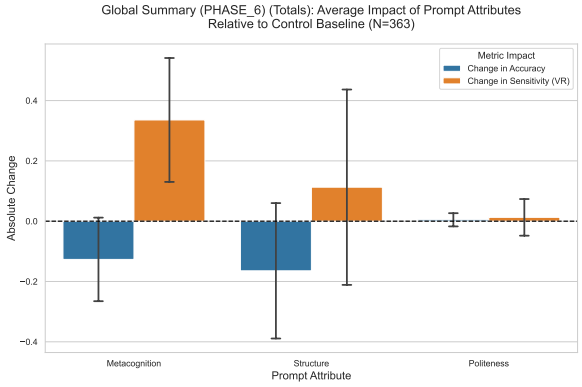


Figure 2: Global summary of the average impact of prompt attributes relative to the Control baseline across all tasks and models. Bars represent the mean absolute change in accuracy and variation ratio (sensitivity), with error bars indicating standard deviation.

mance gains, the aggregate results reflect the significant accuracy drops encountered in multiple-choice tasks. The visualization confirms that Politeness is the most consistently neutral-to-positive intervention, while Metacognition and Structure often introduce instability or severe performance penalties in less capable models.

7 Discussion

Across the full result set, three broader patterns emerge. First, **Politeness** is the safest prompt style. It is almost always neutral or slightly positive, and it never causes the kind of catastrophic failures seen with the other styles. Its strongest result appears on Flan-T5-Large for CoLA, where it improves accuracy by more than six percentage points.

Second, **Metacognition** is the riskiest style. It can help on a binary task such as CoLA for a capable instruction-tuned model, but it consistently harms performance on the multiple-choice tasks and often increases sensitivity sharply. The likely reason is that metacognitive prompts encourage longer, more verbose outputs, which makes answer extraction harder and increases the chance of derailment, especially in smaller or weaker models. Parse rates under metacognition are notably lower than under other styles: for instance, on CoLA with Llama-3.2-1B-Instruct, only about 45% of metacognition responses could be parsed into a valid Yes/No label, compared to over 99% for the control prompt. This means that part of the measured accuracy drop reflects parsing failures rather than incorrect reasoning, though model errors account for the majority of wrong answers even among parsed responses.

Third, **Structure** has mixed effects. On CoLA, structured JSON prompting can be genuinely helpful. On multiple-choice tasks, however, structure often produces the largest accuracy drops. In these cases, lower variation ratio may reflect rigid output behavior rather than robust reasoning.

Furthermore, the computational load of different prompt styles was analyzed. In experiments with Phi-3-Mini on CSQA, the **Structure** prompt (which requires generating a JSON object with reasoning) increased average inference latency by approximately 14.6x compared to the Control prompt. This highlights a significant deployment trade-off: while structured outputs improve parsing reliability and can sometimes improve robustness, they introduce a heavy computational bottleneck, especially in decoder-only causal models where sequence length directly impacts generation time.

These results also suggest that task type matters. The binary CoLA setting tolerates prompt modifications better than the multiple-choice settings. By contrast, CSQA and QASC are both more brittle under prompt changes, and the negative effects are especially large when the prompt encourages extra reasoning or enforces a rigid output schema. Prompt robustness in small models therefore depends not only on the prompt itself, but also on the interaction between model capability and task structure.

These patterns, however, are not uniform across models. The extended results in Appendix B show that for base decoder-only models such as Llama-3.2-1B and Pythia-410M, metacognition causes even larger accuracy drops (-20 to -33 percentage points on CoLA) than those observed for Flan-T5-Large, because these models lack the instruction-following ability needed to produce well-formatted responses. Structured prompting can fail completely in this setting: Llama-3.2-1B produces 0.0% accuracy with a variation ratio of 0.0 under the Structure condition on CoLA as the model generates unparseable output for every input. This is exactly the kind of false stability discussed in the introduction: low variation ratio that reflects rigid failure rather than genuine robustness. Among instruction-tuned decoder-only models, Llama-3.2-1B-Instruct shows a particularly stark split: Politeness achieves near-perfect stability (VR = 0.004) with accuracy comparable to Control, while Metacognition causes a -34.8 pp accuracy drop with VR = 0.807. Phi-3-Mini, the strongest model overall, shows smaller accuracy changes but high output-level instability under both Structure and Metacognition (VR > 0.80). Across all models, Politeness is the only style that is consistently neutral or positive, reinforcing our conclusion that it is the safest intervention. These model-dependent effects suggest that prompt robustness findings should always be conditioned on the model's baseline capability and instruction-following capacity.

8 Limitations

Our study has several limitations. First, although we evaluate six models, only two (Flan-T5-Large and Phi-3-Mini) pass the OOTB screening on all three datasets. Because the main cross-task comparison (Table 4) is based on Flan-T5-Large, the conclusions drawn from it may not generalize to all model families, particularly base decoder-only models. Second, the model suite is intentionally small and local, which is appropriate for the course project but limits generalization to larger frontier models. Third, although we evaluate three benchmarks, they still cover only a narrow slice of possible NLP behavior. All three are classification-style tasks, and the results may not transfer directly to more open-ended generation settings.

Fourth, our perturbation pipeline is controlled but imperfect. WordNet-based synonym replacement and model-based paraphrasing aim to preserve meaning, but neither method guarantees semantic equivalence in every case. Some perturbations may still change the expected result in subtle ways. Fifth, the final reported results aggregate repeated runs across seeds and perturbation settings. This improves stability of the summary, but it doesn't address the finer-grained differences between specific perturbation methods.

Finally, our primary sensitivity metric is computed over raw decoded outputs rather than parsed labels. This choice is defensible because output form is part of prompt behavior, but it also means that some measured instability reflects formatting differences rather than task-level answer flips. We intentionally avoid a primary focus on parsed-label sensitivity because, in restricted classification tasks, it often becomes a mathematical proxy for accuracy; whenever the correct label is the most frequent model response, the variation ratio simplifies to $1 -$ Accuracy, offering little diagnostic value beyond the standard error rate. Raw-output sensitivity, by contrast, captures the stylistic and formatting drift that characterizes prompt instability in smaller models.

9 Conclusion

We presented an inference-only framework for studying prompt robustness in small language models through sensitivity under semantic-preserving perturbations. Using prompt attributes drawn from prior work, metacognition, structure, and politeness, we examined how different kinds of prompt design affect both robustness and task performance on CoLA, QASC, and CSQA. Our findings support a cautious conclusion. Prompt properties do matter, but their effects depend strongly on model capability and task structure. CoLA provides the clearest case in which prompt design can help, while the multiple-choice tasks show that the same prompt changes can also harm performance substantially. For small models, robustness should therefore be evaluated jointly with accuracy, parseability, and output behavior rather than as a standalone property.

AI Usage Disclosure

We used AI-based tools (primarily ChatGPT, Claude, and Gemini) as assistants during this project. These tools were used for language editing, rephrasing drafts for clarity, LaTeX format-

ting, and exploring data analysis approaches such as identifying patterns in results and generating visualization ideas. Script writing and code debugging were also assisted by AI tools.

References

Author Long and 1 others. 2025. What makes a prompt effective? a taxonomy of prompt attributes. *arXiv preprint arXiv:2501.00000*.

Sheng Lu, Hendrik Schuff, and Iryna Gurevych. 2024. How are prompts different in terms of sensitivity? In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 5833–5856, Mexico City, Mexico. Association for Computational Linguistics.

Appendix

A Full Prompt Templates

This appendix lists the prompt templates used in the experiments. The templates differ only in the instruction framing; the underlying task input remains unchanged across prompt styles.

A.1 CoLA Prompt Templates

For CoLA, the model receives a sentence and must answer whether it is grammatically acceptable.

Style	Template

Control	`Is this sentence grammatically correct? Answer Yes or No.`
	`Sentence: "{sentence}"`
	`Answer:`

Metacognition	`Is this sentence grammatically correct? Answer Yes or No.`
	`Before answering, carefully check the grammar rules. Verify your answer is correct.`
	`Sentence: "{sentence}"`
	`Think about it carefully, then answer:`

Structure	`Analyze whether the following sentence is grammatically correct.`
	`Sentence: "{sentence}"`
	`You MUST respond with valid JSON in this exact format:`
	`{"reasoning": "your brief explanation here", "final_answer": "Yes or No"}`
	`(Yes = grammatically correct, No = has error)`
	`Output only the JSON, nothing else.`

Politeness	`Hello! I would really appreciate your help with this.`
	`Could you please tell me if this sentence is grammatically correct?`
	`Please answer with Yes or No.`
	`Sentence: "{sentence}"`
	`Thank you! Your answer:`

Table 5: Full CoLA prompt templates.

A.2 QASC and CSQA Prompt Templates

QASC and CSQA share the same general multiple-choice format. QASC uses answer letters A–H, while CSQA uses answer letters A–E. In QASC, the reported experiments use the question and answer choices without supporting fact injection.

Style	Template
Control	`{question and choices}` `Answer with just the letter (A-H/A-E):`

Metacognition	`{question and choices}`
	`Think carefully about the information provided.`
	`Verify your reasoning before answering.`
	`Answer with just the letter (A-H/A-E):`

Structure	`{question and choices}`
	`You MUST respond with valid JSON in this exact format:`
	`{"reasoning": "your brief explanation here", "final_answer": "X"}`
	`Where X is the letter A-H/A-E. Output only the JSON, nothing else.`

Politeness	`Hello! I would really appreciate your help with this.`
	`{question and choices}`
	`Please provide your answer (just the letter A-H/A-E). Thank you!`

Table 6: Full multiple-choice prompt templates for QASC and CSQA. QASC uses A–H answer labels; CSQA uses A–E answer labels.

B Extended Results Across Models

The main paper reports the cross-task prompt-style comparison for Flan-T5-Large. Here, extended results are provided for the additional model-dataset pairs that passed or were analyzed in the final result pool. Values are reported as task accuracy (%), variation ratio (VR), and parse rate (%). Deltas (Δ) are relative to the Control prompt for the same model and dataset. Parse rate measures the percentage of model responses that were successfully parsed into the expected task format.

B.1 CoLA

Model	Style	Acc. (%)	VR	Parse (%)	Δ Acc.	Δ VR	Δ Parse
Flan-T5-Large	Control	63.7	0.1621	100.0	–	–	–
	Metacognition	67.7	0.1693	100.0	+3.9	+0.0072	+0.0
	Structure	70.3	0.1344	100.0	+6.5	-0.0277	+0.0
	Politeness	69.9	0.1778	100.0	+6.2	+0.0158	+0.0
Llama-3.2-1B	Control	68.7	0.4153	100.0	–	–	–
	Metacognition	48.6	0.6413	63.0	-20.2	+0.2260	-37.0
	Structure	0.0	0.0000	0.0	-68.7	-0.4153	-100.0
	Politeness	68.7	0.4881	100.0	+0.0	+0.0728	+0.0
Llama-3.2-1B-Instruct	Control	68.5	0.0498	100.0	–	–	–
	Metacognition	33.7	0.8074	45.1	-34.8	+0.7575	-54.9
	Structure	51.3	0.4527	99.9	-17.2	+0.4029	-0.1
	Politeness	68.7	0.0043	99.9	+0.2	-0.0456	-0.1
Phi-3-Mini	Control	76.0	0.2750	100.0	–	–	–
	Metacognition	76.4	0.8032	100.0	+0.4	+0.5282	+0.0
	Structure	79.2	0.8226	99.7	+3.2	+0.5475	-0.3
	Politeness	76.0	0.1841	100.0	+0.0	-0.0909	+0.0
Pythia-410M	Control	59.8	0.5680	86.8	–	–	–
	Metacognition	26.6	0.7937	37.8	-33.2	+0.2257	-49.0
	Structure	62.6	0.3686	82.9	+2.7	-0.1994	-3.9
	Politeness	57.9	0.5250	79.5	-2.0	-0.0431	-7.3

Table 7: Extended CoLA results across models and prompt styles.

B.2 QASC

Model	Style	Acc. (%)	VR	Parse (%)	Δ Acc.	Δ VR	Δ Parse
Flan-T5-Base	Control	44.7	0.2625	99.8	–	–	–
	Metacognition	41.5	0.5915	94.9	-3.3	+0.3290	-4.9
	Structure	39.7	0.2693	89.7	-5.0	+0.0067	-10.1
	Politeness	45.3	0.2708	99.8	+0.5	+0.0082	+0.0
Flan-T5-Large	Control	62.2	0.2345	99.4	–	–	–
	Metacognition	36.9	0.6529	84.3	-25.2	+0.4184	-15.1
	Structure	22.1	0.1428	24.2	-40.0	-0.0917	-75.2
	Politeness	62.2	0.2377	99.7	+0.0	+0.0032	+0.3
Phi-3-Mini	Control	78.7	0.3295	100.0	–	–	–
	Metacognition	77.2	0.5792	99.2	-1.5	+0.2497	-0.8
	Structure	70.0	0.8177	97.2	-8.7	+0.4882	-2.8
	Politeness	78.7	0.3954	98.9	+0.0	+0.0660	-1.1

Table 8: Extended QASC results across models and prompt styles. QASC is evaluated without fact injection.

B.3 CSQA

Model	Style	Acc. (%)	VR	Parse (%)	Δ Acc.	Δ VR	Δ Parse
Flan-T5-Base	Control	72.3	0.2487	99.9	–	–	–
	Metacognition	69.3	0.5594	99.8	-2.9	+0.3107	-0.1
	Structure	65.6	0.2512	94.3	-6.7	+0.0026	-5.6
	Politeness	70.5	0.2478	99.9	-1.8	-0.0008	+0.0
Flan-T5-Large	Control	83.2	0.2242	99.1	–	–	–
	Metacognition	59.9	0.7021	93.1	-23.3	+0.4780	-6.0
	Structure	50.4	0.2237	48.8	-32.8	-0.0004	-50.3
	Politeness	83.4	0.2170	99.9	+0.2	-0.0071	+0.8
Phi-3-Mini	Control	73.5	0.2731	100.0	–	–	–
	Metacognition	72.3	0.4321	100.0	-1.2	+0.1590	0.0
	Structure	70.5	0.8810	99.6	-3.0	+0.6079	-0.4
	Politeness	74.2	0.4023	99.9	+0.6	+0.1292	-0.1

Table 9: Extended CSQA results across models and prompt styles.

B.4 Key Observations from Extended Results

Several key patterns emerge from the extended cross-model and cross-dataset evaluation:

1. **Structured Prompt Fragility:** Base models, particularly decoder-only architectures like Llama-3.2-1B and Pythia-410M, show catastrophic failures when tasked with structured JSON output. Llama-3.2-1B (Base) failed to produce a single parseable JSON response for CoLA (Δ Parse -100.0). Even instruction-tuned models like Flan-T5-Large show significant degradation in parseability on multiple-choice tasks (QASC/CSQA) under structured constraints, which accounts for the majority of their accuracy loss in those settings.

2. **Metacognition and Induced Verbosity:** Instructions that encourage reasoning (e.g., "Think carefully") tend to increase output verbosity. While this can slightly improve task performance for some models (e.g., Flan-T5-Large on CoLA), it often destroys parseability for weaker models. For Pythia-410M on CoLA, the metacognition style led to a 49.0 percentage point drop in parse rate as the model shifted from binary labels to discursive text. For example, instead of a "Yes/No" label, Pythia-410M often repeated the input sentence multiple times or generated a list of numbered repetitions.[1] Instruction-tuned models like Llama-3.2-1B-Instruct also exhibited this failure mode, providing detailed grammatical analyses while omitting the required "Yes" or "No" label entirely.[2]

3. **Stability-Accuracy Trade-off:** Forced structure can sometimes reduce the variation ratio (VR), suggesting higher stability. However, this stability is often "empty" if the model cannot accurately follow the task or the format. For example, Llama-3.2-1B-Instruct on CoLA showed a sharp increase in VR variability under the Structure prompt (+0.40) while suffering a 17.2% drop in accuracy.

4. **Instruction Tuning Resilience:** Models that have undergone instruction tuning (Llama-3.2-1B-Instruct, Phi-3-Mini) are generally much more resilient to prompt style interventions. They maintain higher parse rates and more consistent accuracy across diverse styles compared to their base or smaller counterparts.

[1] See for example row 6,451 in `outputs/results/phase_6/pythia-410m/cola/synonym/w1_n20_s105_detail.csv`.

[2] See for example row 10,505 in `outputs/results/phase_6/llama-3.2-1b-instruct/cola/paraphrase/n20_s105_detail.csv`.

C Evaluation and Parsing Details

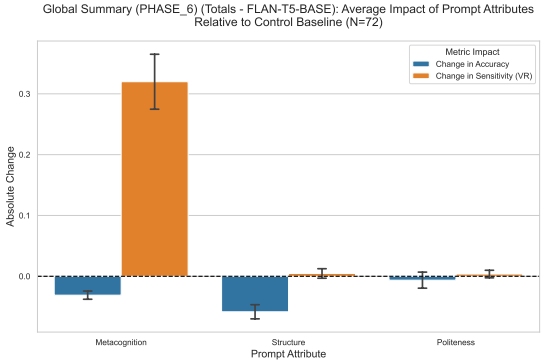
Table 10 summarizes the evaluation rules used in the final experiments. These rules are important because sensitivity and accuracy measure different aspects of model behavior.

Component	Rule
OOTB screening	Each model-dataset pair is first evaluated with the Control prompt on unperturbed examples. If the model fails the dataset-specific threshold, sensitivity results are not treated as meaningful evidence.
OOTB denominator	Accuracy is computed over all sampled items. Unparseable outputs count as incorrect rather than being removed from the denominator.
Task accuracy	Accuracy is computed over parsed task labels: A–H for QASC, A–E for CSQA, and Yes/No for CoLA.
Structured parsing	For Structure prompts, the parser first attempts to read the `final_answer` field from the JSON output. Markdown code fences are stripped before parsing when present.
CoLA parsing	For CoLA, the parser uses final-answer-oriented rules: explicit answer patterns, Yes/No near the end of the response, grammaticality keywords near the end of the response, and a fallback scan for bare Yes/No tokens.
Variation ratio	The main reported VR is computed over raw decoded strings after minimal normalization. This means that formatting and response-style changes are counted as output instability.
Parsed-label VR	The implementation also supports computing VR over parsed labels, but this is not the main setting reported in the paper.

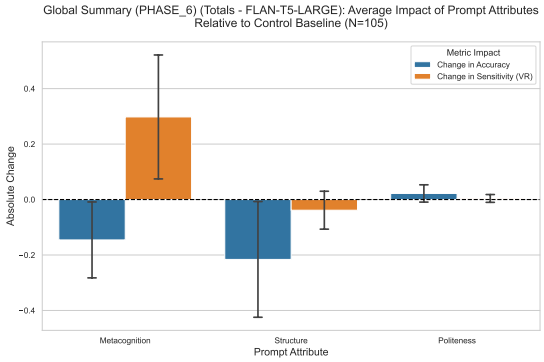
Table 10: Summary of final evaluation and parsing rules.

D Detailed Impact Plots

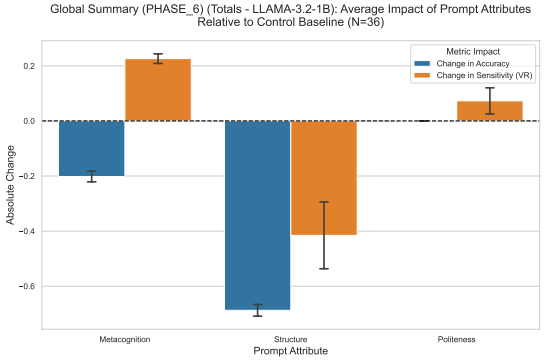
This appendix provides a comprehensive visualization of the impact of prompt attributes across all evaluated models. The following plots show the average absolute change in Accuracy and Sensitivity (Variation Ratio) relative to the Control baseline for each model, aggregating results across datasets and perturbation types. These summaries provide a global view of how metacognition, structure, and politeness influence performance and stability across the entire experimental suite.



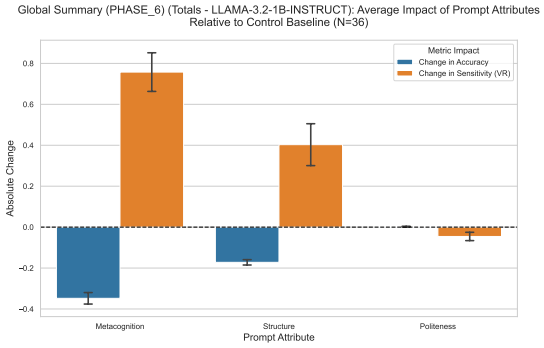
Flan-T5-Base



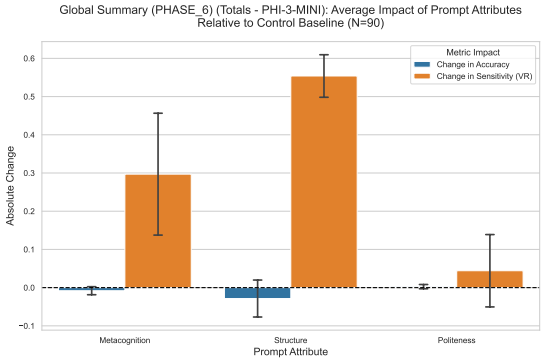
Flan-T5-Large



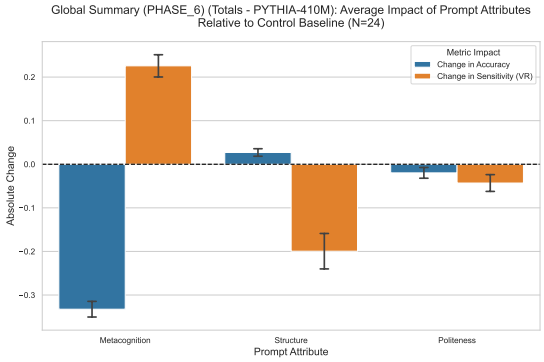
Llama-3.2-1B



Llama-3.2-1B-Instruct



Phi-3-Mini



Pythia-410M

Figure 3: Aggregate summary impact plots per model.